Task -8:- Technical Documentation(AWS RDS Setup and Grafana Cloud Dashboards for Data Visualization)

1. Introduction

1.1 Purpose

The purpose of this task is to implement a cloud-based database monitoring and visualization system using **AWS RDS** and **Grafana Cloud**.

The implementation demonstrates:

- Managed database deployment
- Infrastructure monitoring
- Database analytics visualization
- Performance monitoring using automation-generated workload

1.2 Scope

This project includes:

AWS RDS database deployment
Dataset migration to RDS
Grafana Cloud integration
Infrastructure monitoring dashboards
Database analytics dashboards
Performance Insights visualization
Automated database load simulation

Out of Scope:

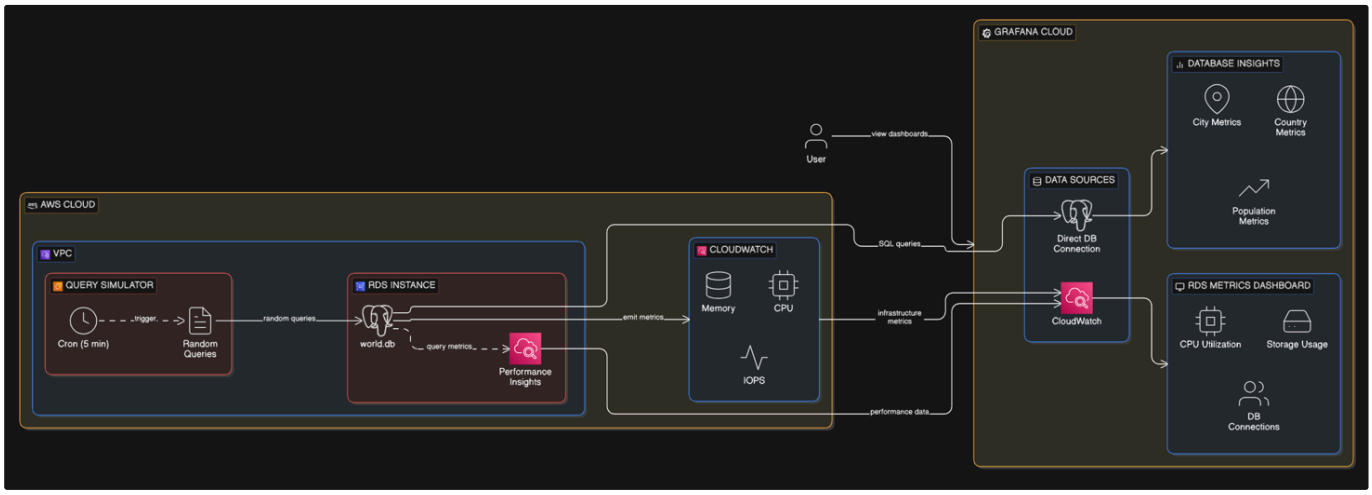
- Production HA architecture
- Multi-region replication
- Application backend deployment

---

2. Architecture Overview

System Architecture Flow

```
Dataset → AWS RDS → CloudWatch Metrics
                        ↓
            Performance Insights
                        ↓
               Grafana Cloud
                        ↓
                Dashboards
```



---

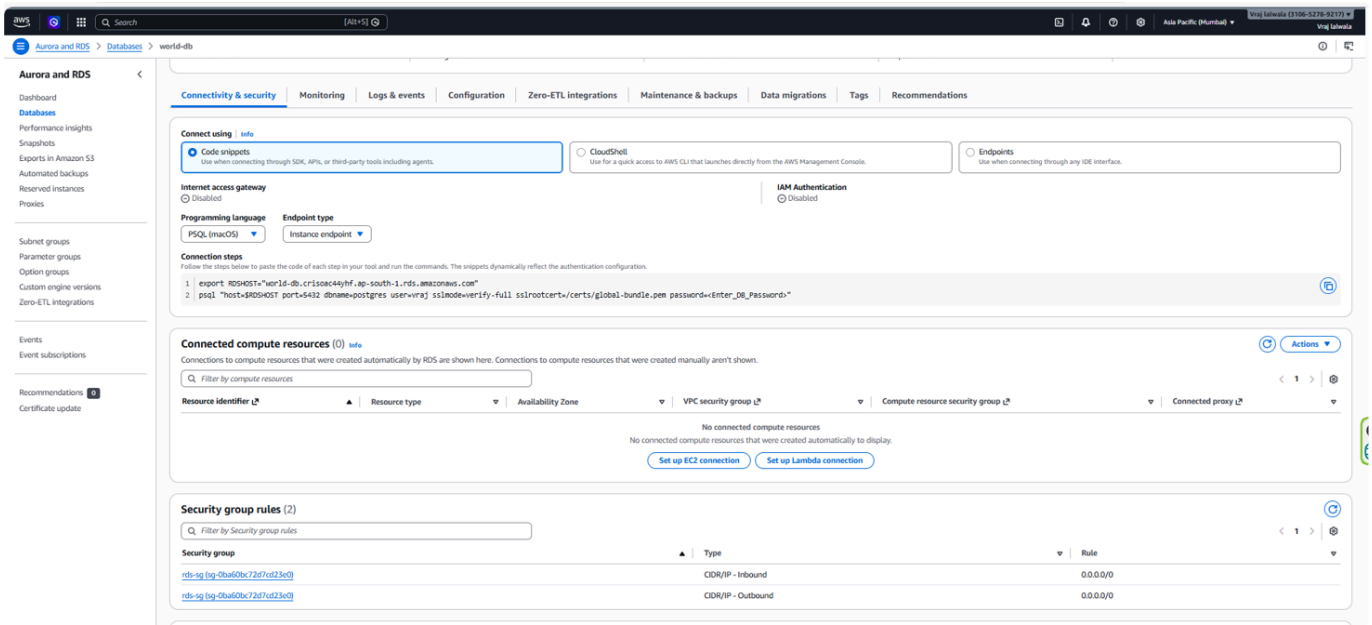
3. AWS RDS Setup

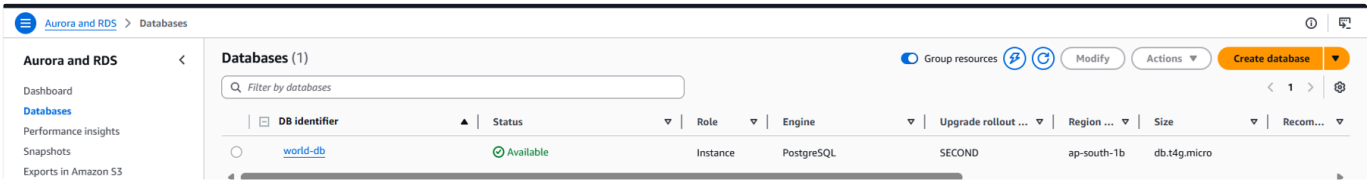
3.1 RDS Instance Creation

An Amazon RDS PostgreSQL instance was created using AWS Free Tier configuration.

Configuration Details

Parameter	Value
Engine	PostgreSQL
Instance Type	db.t3.micro
Storage	20 GB
Public Access	Enabled
Monitoring	Performance Insights Enabled
Region	ap-south-1



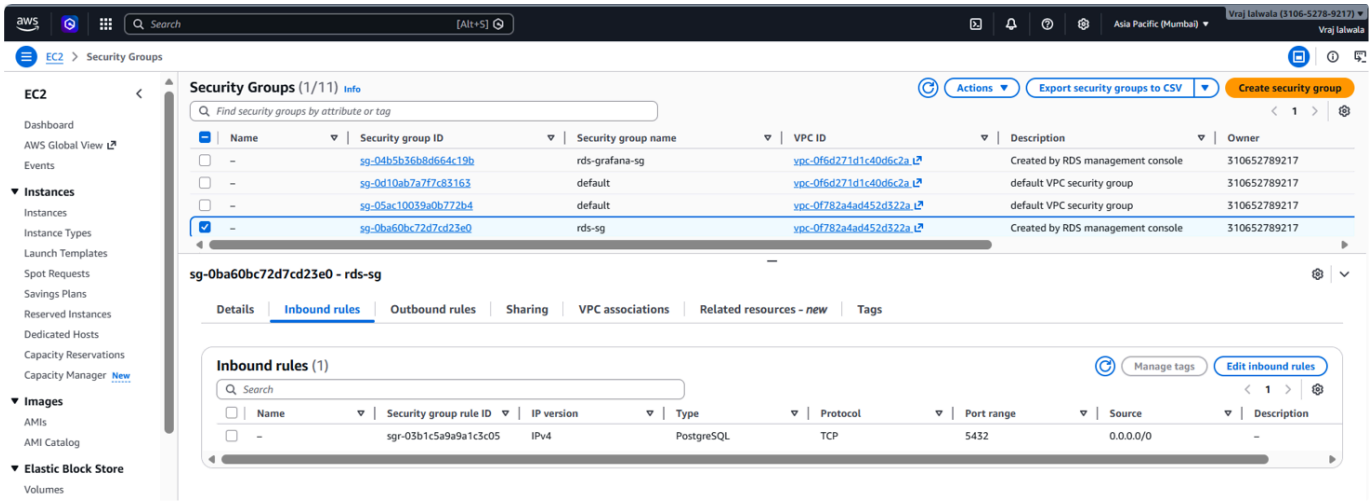


3.2 Security Configuration

Security group rules were configured to allow database connectivity.

Inbound Rule:

Type	Port	Source
PostgreSQL	5432	0.0.0.0/0



4. Dataset Import into RDS

4.1 Dataset Used

Dataset Source:

```
world.db (Gramener Dataset Repository)
```

Contains:

- Country data
- City information
- Population statistics
- Language information

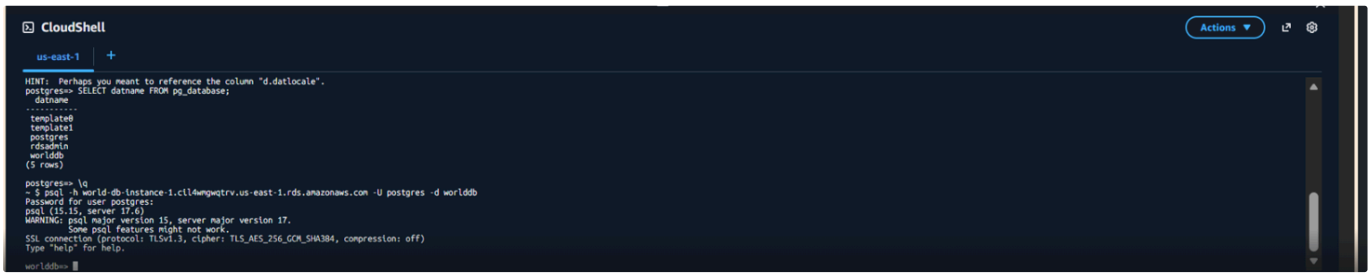
4.2 Data Migration Process

Steps performed:

1. Download dataset in AWS CloudShell.
2. Convert SQLite database to SQL dump.
3. Clean incompatible SQLite syntax.
4. Import data into PostgreSQL RDS instance.
5. Validate imported tables.

Tables Created:

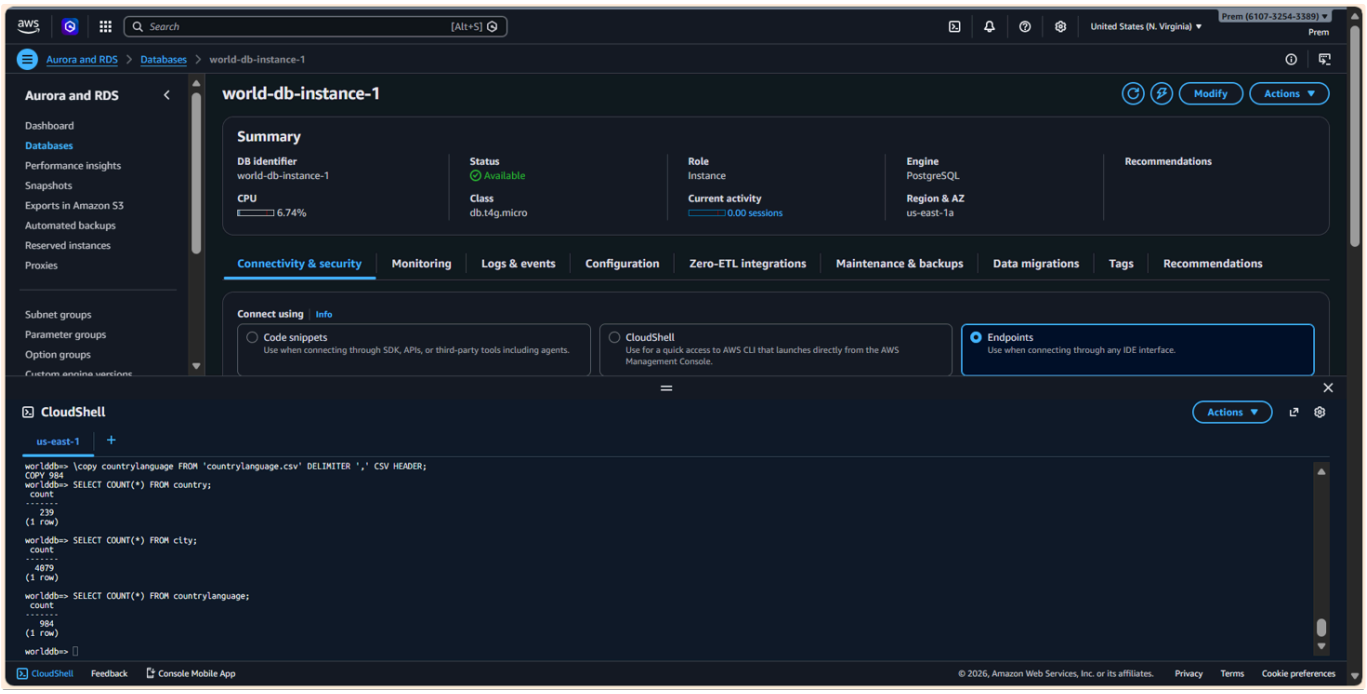
- city
- country
- countrylanguage



4.3 Data Validation

Sample validation queries executed:

```
SELECT COUNT(*) FROM city;
SELECT COUNT(*) FROM country;
```



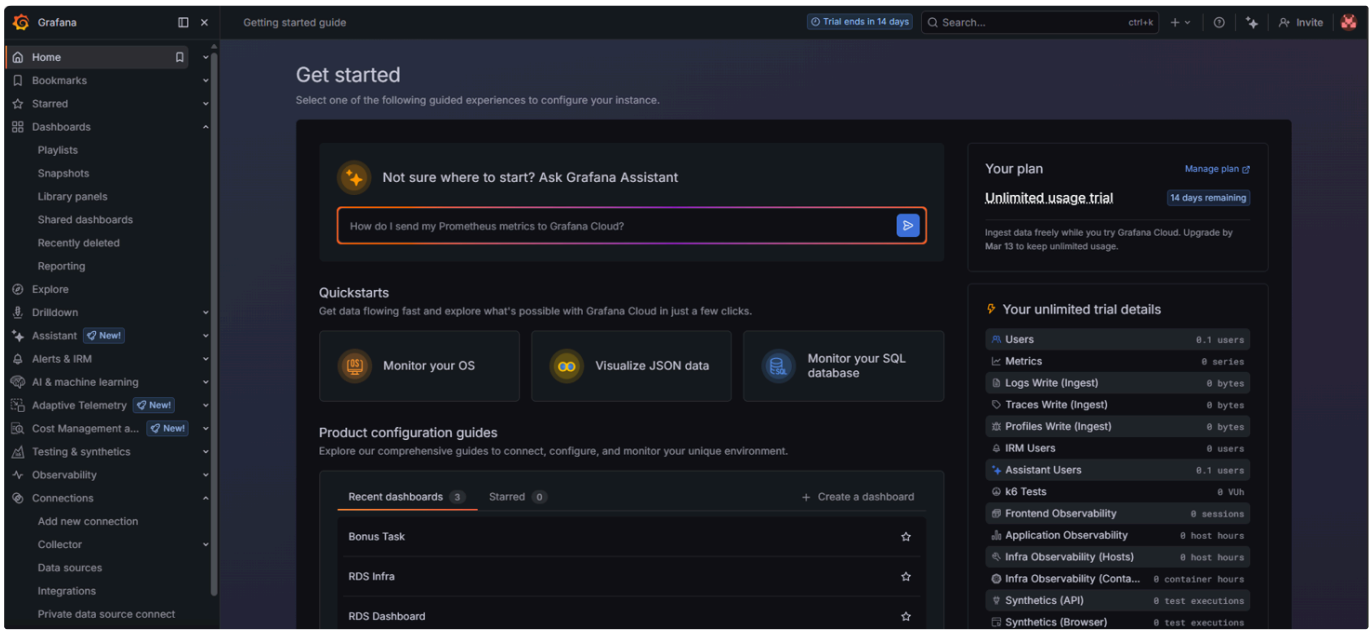
5. Grafana Cloud Configuration

5.1 Grafana Cloud Setup

A Grafana Cloud free-tier account was created for centralized monitoring.

Configuration included:

- Stack creation
- Dashboard workspace setup



5.2 CloudWatch Integration

AWS CloudWatch datasource was added to Grafana for infrastructure monitoring.

Integrated Metrics:

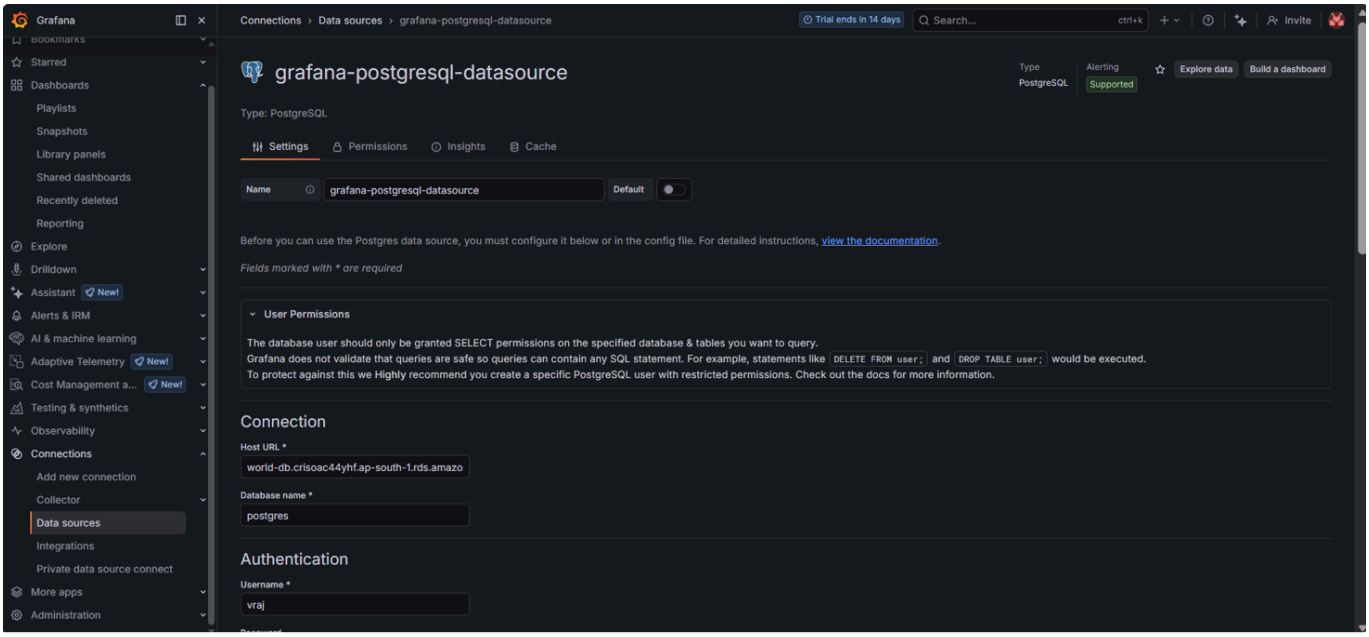
- CPU Utilization
- Storage
- Database Connections
- IOPS Metrics

5.3 PostgreSQL Datasource Configuration

Direct database connectivity established between Grafana and AWS RDS.

Connection Parameters:

- RDS Endpoint
- Database Name
- Credentials
- Port 5432



6. Dashboard Implementation

6.1 Dashboard 1 — RDS Infrastructure Monitoring

This dashboard monitors database infrastructure performance.

Metrics Visualized

- CPU Utilization
- Database Connections
- Read IOPS
- Write IOPS
- Free Storage Space

Purpose:

Monitor health and resource utilization of RDS instance.

6.2 Dashboard 2 — Database Analytics Dashboard

SQL queries were used to visualize dataset insights.

Panels Created

Total Cities

```
SELECT COUNT(*) FROM city;
```

Country-wise City Distribution

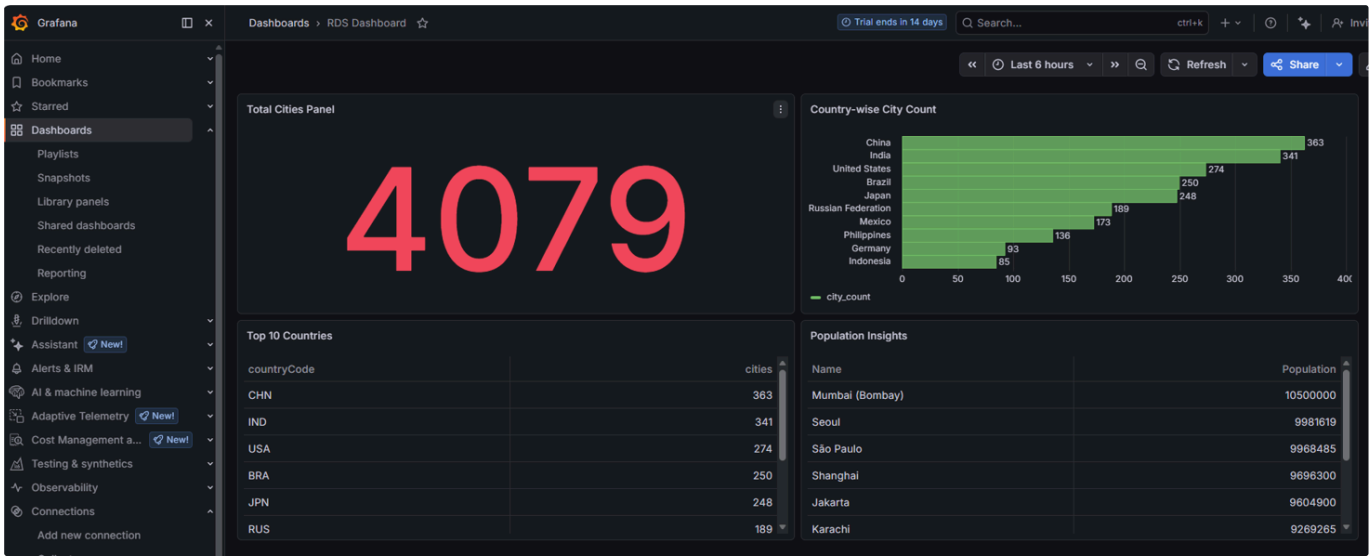
```
SELECT "countryCode", COUNT(*)
FROM city
GROUP BY "countryCode";
```

Top Populated Cities

```
SELECT "Name","Population"
FROM city
ORDER BY "Population" DESC
LIMIT 10;
```

Purpose:

Provide analytical insights from stored dataset.



6.3 Dashboard 3 — Performance Insights Dashboard

Displays runtime database performance metrics.

Metrics Included:

- Database Load
- Active Connections
- CPU Utilization Trends
- Query Activity

Purpose:

Analyze database workload behavior.

7. Bonus Task — Automation & Query Load Simulation

7.1 Objective

Simulate real-world database workload to generate measurable performance metrics.

7.2 Automation Script

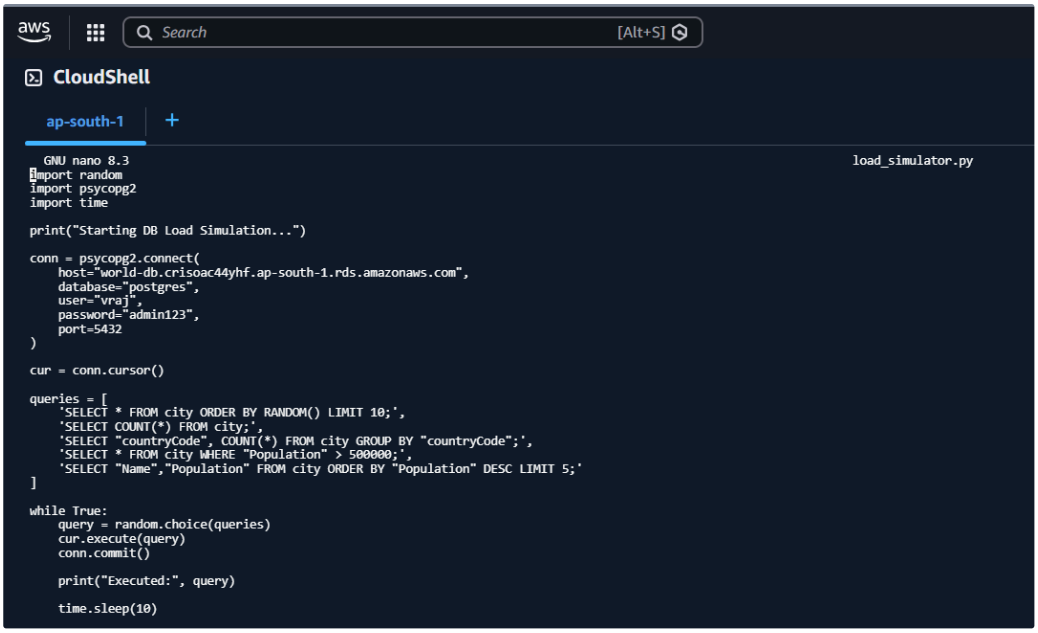
A Python automation script was created in AWS CloudShell.

Functionality:

- Executes randomized SQL queries
- Performs aggregations and filtering
- Generates continuous database activity

Query Types:

- Random city lookup
- Country aggregation
- Population filtering

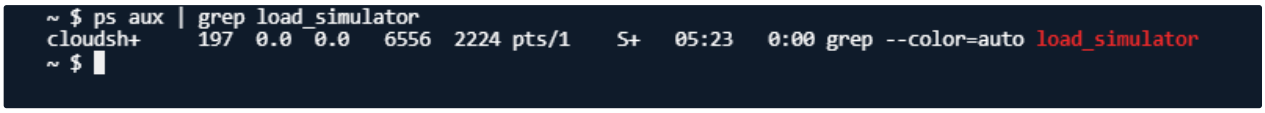


7.3 Scheduling Mechanism

Since AWS CloudShell does not support cron services, automation was implemented using background execution:

```
nohup python3 load_simulator.py &
```

The script executes queries every 5 minutes using timed intervals.



7.4 Performance Verification

Database workload impact was verified using AWS Performance Insights.

Observed Results:

- Increased DB Load
- Active sessions
- Query execution spikes

---

8. Challenges Faced

- RDS connectivity issues due to subnet configuration
- SQLite to PostgreSQL migration incompatibility
- Case-sensitive column conflicts
- CloudShell cron limitation

Solutions involved:

- Security group modification
- SQL cleanup
- Identifier normalization
- Background automation execution

---

9. Result

The system successfully achieved:

Managed database deployment
Real-time infrastructure monitoring
Data analytics visualization
Automated workload simulation
Performance monitoring via Grafana

---

10. Conclusion

This implementation demonstrates a complete cloud-native database observability workflow integrating AWS RDS, CloudWatch, Performance Insights, and Grafana Cloud.

The solution provides both operational monitoring and analytical visualization capabilities aligned with modern DevOps practices.

---

11. Customer Acknowledgement

Name: _____
Title: _____
Signature: _____
Date: _____